

ExpenseVault: CLI-Based Personal Finance Tracker in Go

Project Title: ExpenseVault – A Privacy-Focused Personal Finance CLI Tool with Interactive TUI and Backend Sync

Duration: 2-3 Months

Language: Go (Golang)

Key Features in Detail

- **Transaction Management:** Add, list, edit, and delete transactions with details like amount, category, date, and notes.
- **Categorization & Rules:** Manual or rule-based categorization (e.g., keyword matching in notes).
- **Reporting:** Monthly/weekly summaries, income vs. expense breakdowns, balance trends, and category-wise spending.
- **CSV Import/Export:** Parse standard bank CSV files for bulk import and export reports/transactions.
- **Interactive TUI:** Beautiful terminal dashboard with tables, searchable lists, menus, and simple ASCII charts using BubbleTea.
- **Local Storage:** Persistent data using SQLite.
- **Backend Sync (Advanced):** Self-hosted REST API with JWT authentication and `sync` command for multi-device use.
- **Extras:** PDF/CSV report export, password hashing (bcrypt), comprehensive error handling, and unit tests.

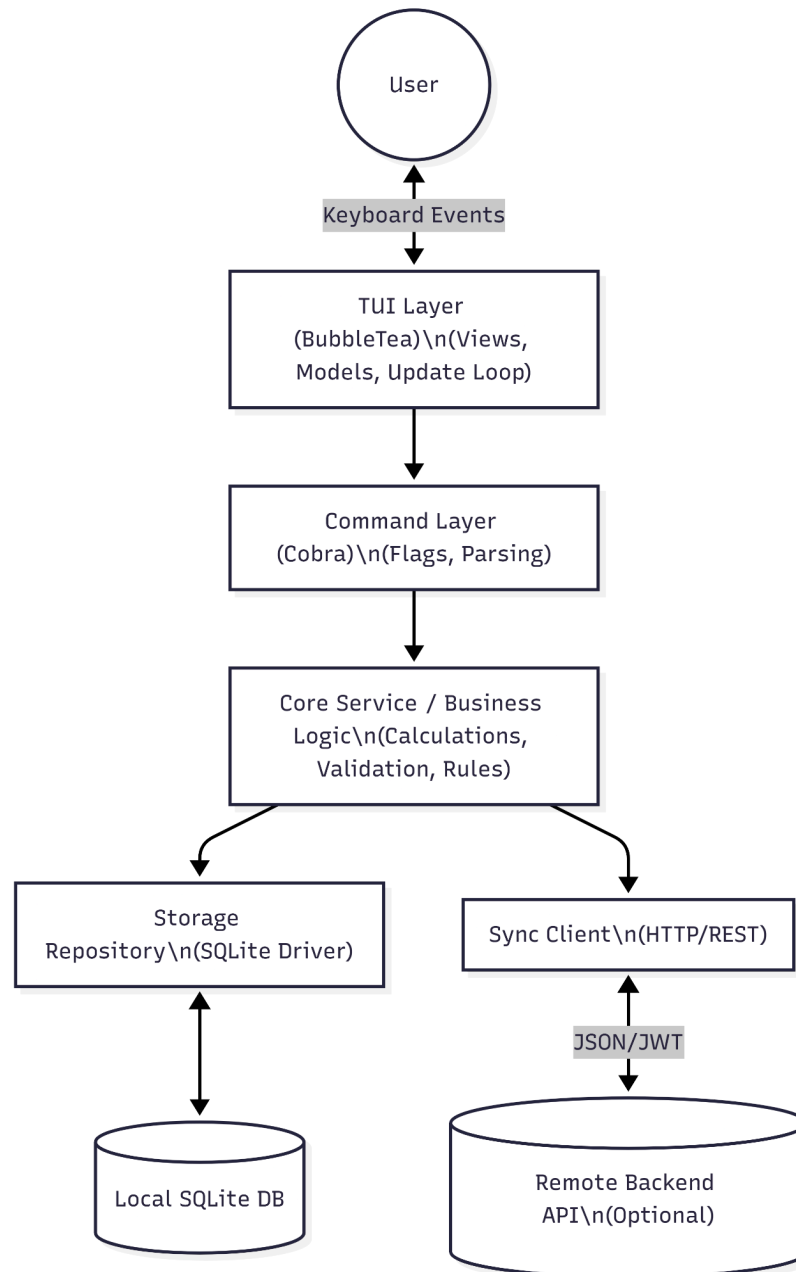
Benefits of Using CLI

- Fast, lightweight, and keyboard-driven – ideal for quick daily entries.
- Works offline with full privacy – data stays on your device by default.
- Scriptable and automatable (e.g., cron jobs for reports).
- Low resource usage and cross-platform compatibility.
- Focused, distraction-free experience compared to web apps.

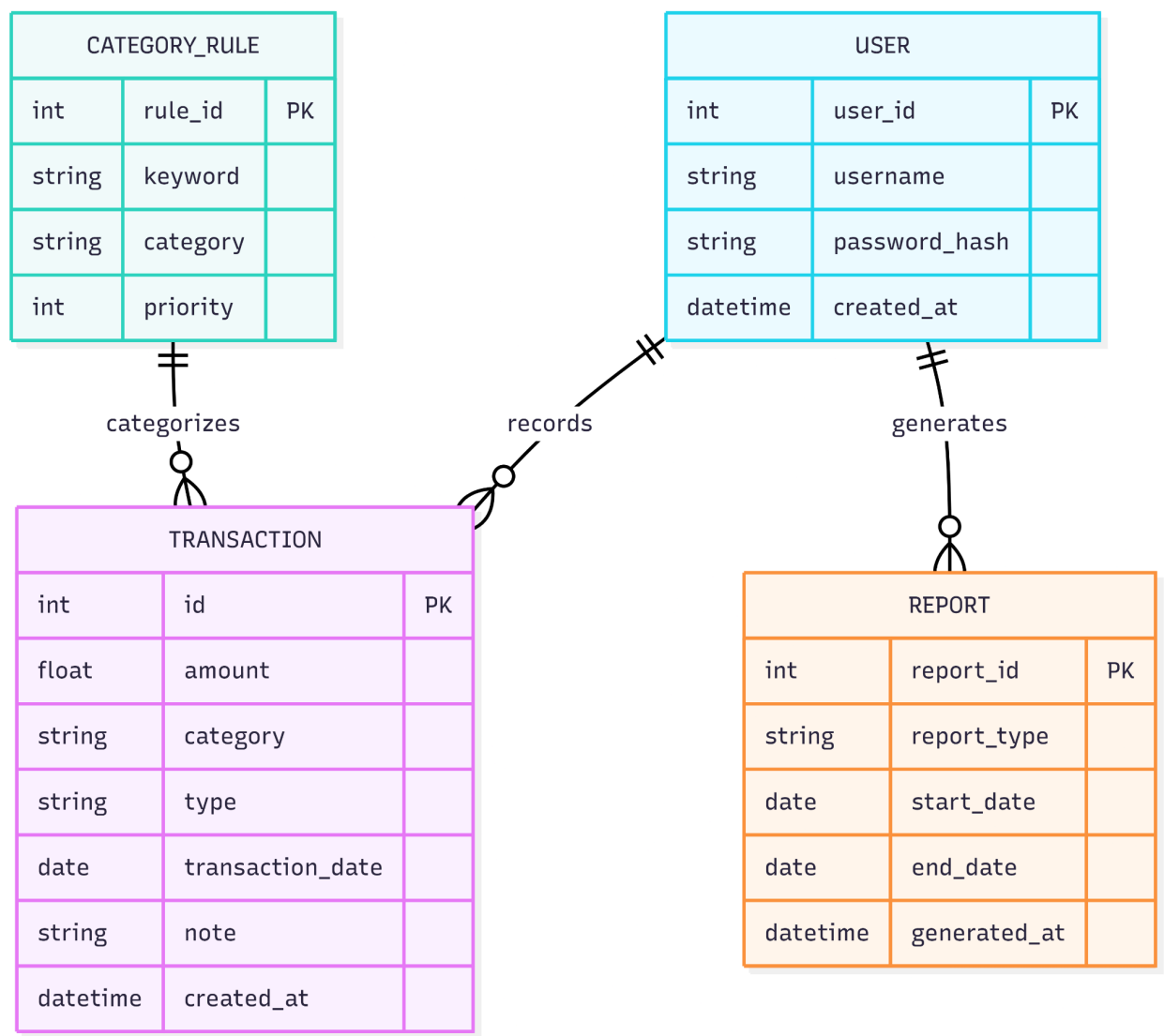
Benefits of Using Go for This CLI Project

- Compiles to a single static binary – easy to distribute and run anywhere without dependencies.
- Excellent for CLI tools (used in Docker, Kubernetes, Hugo, Cobra).
- Built-in concurrency for responsive TUI and background tasks.
- Strong standard library (CSV, JSON, crypto) and memory safety.
- Fast execution and great testing/benchmarking support.
- Clean, readable code – perfect for learning and maintainability.

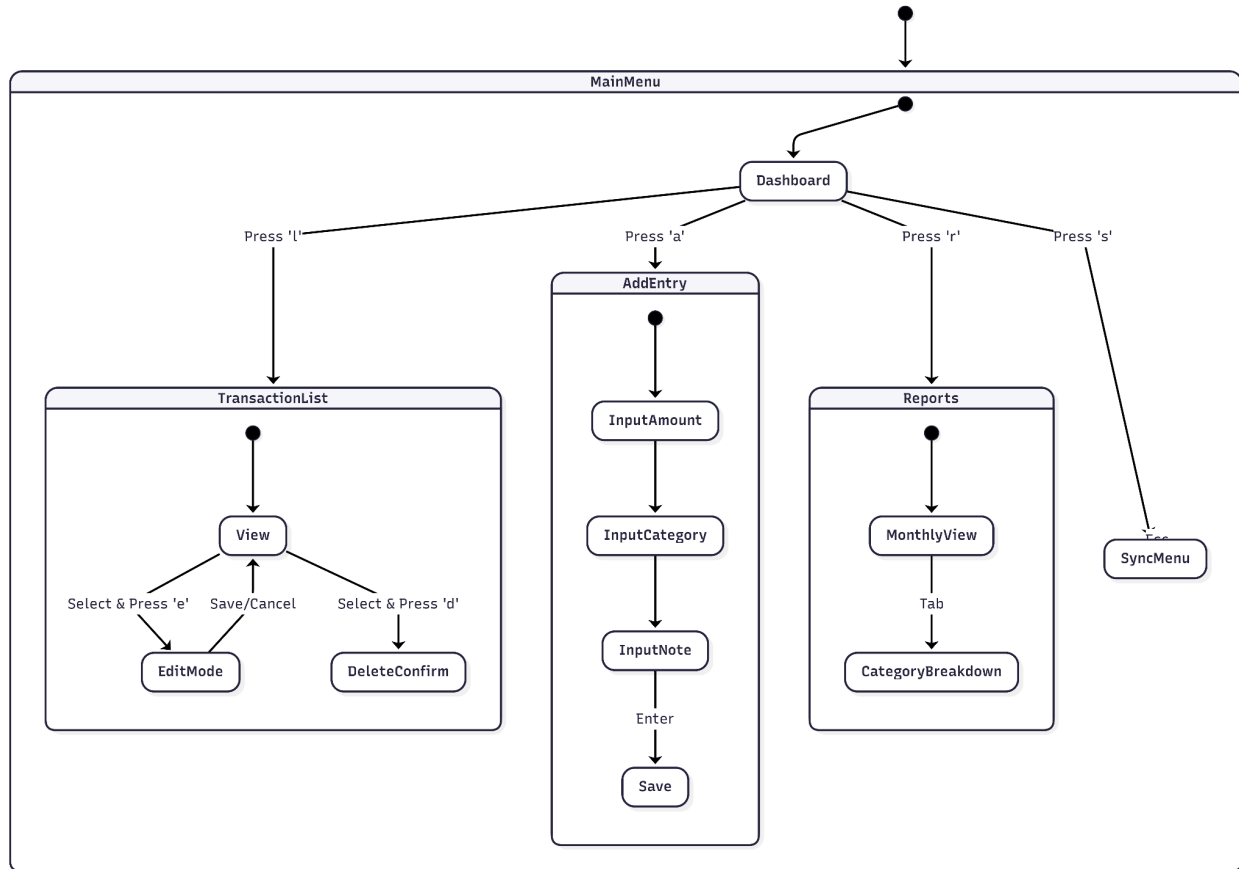
ExpenseVault Architecture Flow User Interaction CLI



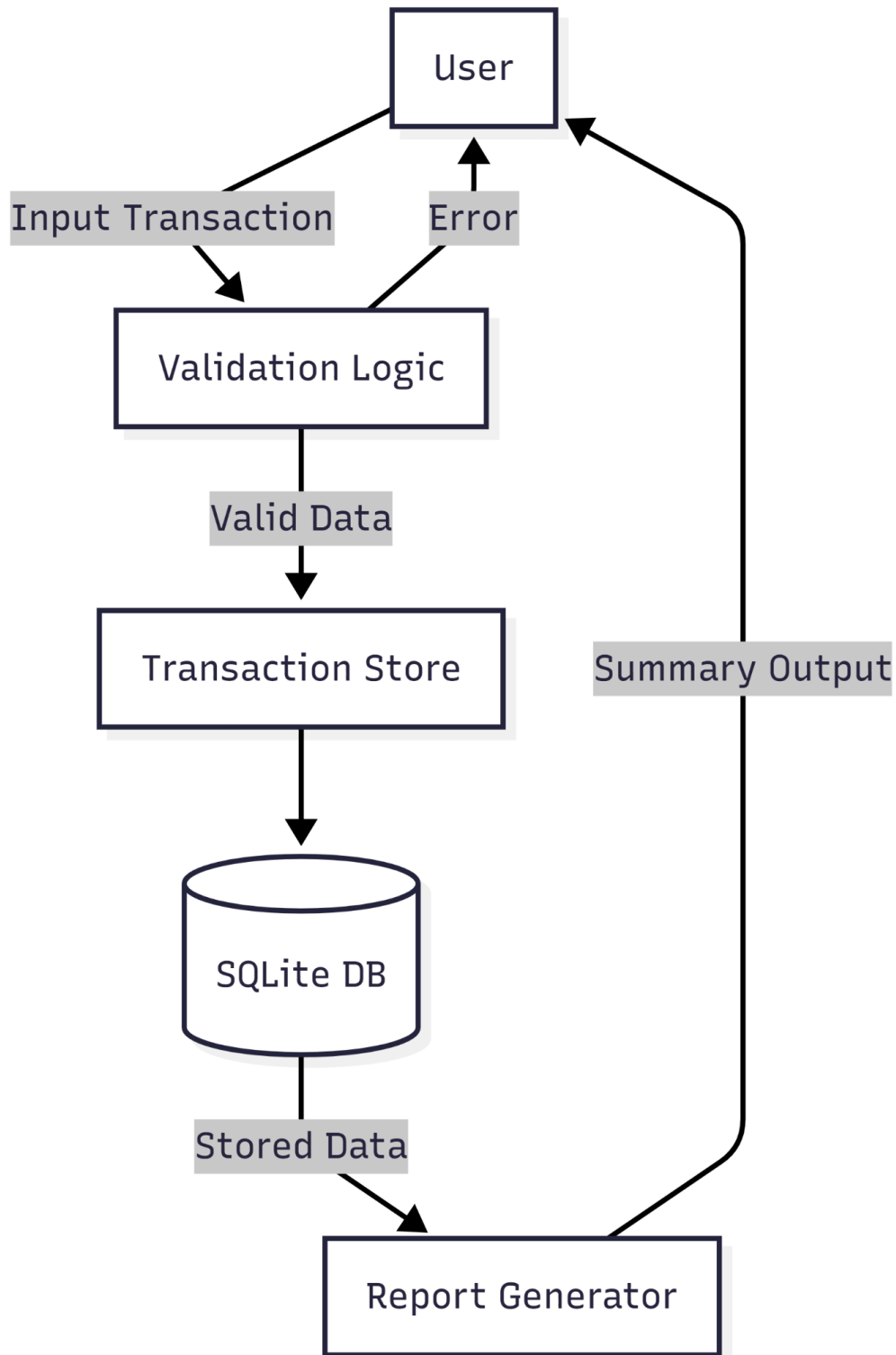
ExpenseVault ERD



ExpenseVault UI flow diagram



ExpenseVault Data Flow Diagram



Syllabus Lab Exercises Integration

Below is how each lab exercise from the course will be directly implemented and demonstrated in ExpenseVault.

LAB: Variables, Values and Type + Looping Structures and Control Flow

Implementation:

- Define transaction fields using various types (float64 for amount, string for category/note, time.Time for date).
- Use short declaration (`:=`) and `var` keyword throughout.
- Demonstrate zero values when initializing structs.
- Use `fmt` package for pretty-printing transactions and reports.
- Implement control flow with `if-else/switch` for input validation and categorization rules.
- Use `for`, `for range`, and conditional loops to iterate over transactions for listing and report calculations.

LAB 1: Arrays and Slices

Implementation:

- Core transaction storage as a slice: `var transactions []Transaction`.
- Use composite literals to initialize test data.
- Append new transactions with `append()`.
- Delete transactions by slicing and re-assigning (e.g., `transactions = append(transactions[:i], transactions[i+1:]...)`).
- Slice operations for filtering (e.g., recent transactions, monthly view).
- Use `make()` for pre-allocating slices in report generation.
- Demonstrate multi-dimensional slices for grouped reports (e.g., `[][]float64` for monthly category totals).

- Contrast with fixed-size arrays for small static lists (e.g., default categories array).

LAB 2: Map and Structs

Implementation:

- Define `Transaction` struct with fields and tags (for future JSON/DB).
- Use embedded structs (e.g., embed `time.Time`).
- Create maps for fast lookups and aggregation: `map[string]float64` for category totals, `map[time.Month][]Transaction` for monthly grouping.
- Add/update/delete map entries when generating reports.
- Range over maps to display category breakdowns.

LAB 3.1: Functions and Error Handling

Implementation:

- Write reusable functions (variadic for bulk add, defer for file/DB cleanup).
- Implement comprehensive error checking and custom error types (e.g., invalid amount, parse errors).
- Use `defer`, `panic`, and `recover` in critical sections (e.g., TUI crash recovery).
- Proper error logging and user-friendly messages.

LAB 3.2: Interface

Implementation:

- Define interfaces like `Reporter` or `Exporter` with methods (e.g., `Generate() string`).
- Implement multiple report types (monthly, category, yearly) satisfying the same interface for polymorphism.
- Use interfaces in TUI menu to switch between different views/reports seamlessly.

LAB 4: Pointers, Call by Value and Call by Function

Implementation:

- Pass pointers to functions for modifying transactions (e.g., edit function).
- Demonstrate method sets: value vs. pointer receivers (e.g., `String()` on `Transaction`).
- Return pointers from factory functions or DB queries.
- Explicitly show difference between pass-by-value (copy) and pass-by-pointer (mutation).

LAB 4.1: JSON Marshal and Unmarshal + Unit Test Case

Implementation:

- Marshal/unmarshal transactions to/from JSON for backup/export and sync with backend.
- Write table-driven unit tests for marshal/unmarshal edge cases.
- Test report functions and input validation with testify or standard testing package.
- Include benchmarks for large transaction sets.

LAB 5.1: Concurrency

Implementation:

- Use WaitGroups, mutex/atomic for safe concurrent access (e.g., background sync).
- Demonstrate race condition with unsafe map access, then fix with mutex/atomic.

LAB 5.2: Goroutines and Channels

Implementation:

- In TUI: Run user input handling in separate goroutine, communicate updates via channels.
- Background tasks like CSV import or auto-sync using goroutines.
- Use directional channels, `range` over channels, and `select` for timeout/input handling.
- Make UI responsive by decoupling heavy computations (report generation) into goroutines.

Technology Stack

Layer / Component	Technology	Purpose in Project	Justification
Programming Language	Go (Golang)	Core development of CLI application	Compiled language with strong typing, excellent standard library, built-in concurrency, and ideal support for CLI tools
CLI Framework	Cobra	Command parsing and CLI structure	Industry-standard Go CLI framework; enables structured commands, flags, and help menus
Terminal UI (TUI)	BubbleTea	Interactive terminal dashboards	Supports reactive terminal interfaces, tables, menus, and charts using Go
Database	SQLite	Local persistent storage	Lightweight, file-based database suitable for offline-first applications
ORM / DB Access	database/sql + go-sqlite3	Database interaction layer	Uses Go standard library with minimal abstraction for transparency and performance
Data Serialization	JSON	Backup, export, and sync	Native Go support using <code>encoding/json</code>

CSV Handling	encoding/csv	Import/export of bank statements	Part of Go standard library; reliable and fast
Authentication (Sync)	JWT	Secure backend communication	Stateless authentication suitable for distributed systems
Backend (Optional)	Go (net/http)	REST API for sync	Same language stack simplifies maintenance
Concurrency	Goroutines, Channels	Background tasks and UI responsiveness	Core Go feature for concurrent execution
Version Control	Git	Source code management	Industry standard
Build & Distribution	Go toolchain	Build static binaries	Single-binary deployment across platforms